

AWS Cost Optimization

—> What is AWS Cost Optimization?

AWS Cost Optimization means **reducing unnecessary AWS spending** while still keeping:

- Good performance
- High availability
- Security

In simple words: **Use only what you need, pay less, and avoid waste.**

—> Why Cost Optimization is Important

- AWS is **pay-as-you-go** → wrong setup = high bill
- Helps companies **save money**
- Avoids **unused resources**
- Makes cloud usage **efficient**

—> Pillars of AWS Cost Optimization

AWS follows **Cost Optimization pillar** from Well-Architected Framework.

Main goals:

- Right-size resources
- Use cheaper pricing models
- Remove unused services
- Monitor spending regularly

—> Common AWS Services That Increase Cost

- EC2 instances
- EBS volumes & snapshots
- Load Balancers
- RDS databases
- Data transfer
- CloudWatch logs



EC2 instance types

	General Purpose		Compute Optimized	Memory Optimized		Accelerated Computing	Storage Optimized		
Type	t2	m5	c5	r4	x1e	p3	h1	i3	d2
Description	Burstable, good for changing workloads	Balanced, good for consistent workloads	High ratio of compute to memory	Good for in-memory databases	Good for full in-memory applications	Good for graphics processing and other GPU uses	HDD backed, balance of compute and memory	SDD backed, balance of compute and memory	Highest disk ratio
Mnemonic	t is for tiny or turbo	m is for main or happy medium	c is for compute	r is for RAM	x is for xtreme	p is for pictures	h is for HDD	i is for IOPS	d is for dense

—> EC2 Cost Optimization

a) Right-Sizing EC2

- Do not use **large instance** if small is enough
- Example:
 - Instead of `t3.large` → use `t3.micro`

Right-sizing means **choosing the correct EC2 instance size** based on actual usage.

- Do not use a **large instance** if CPU and memory usage is low
- Check **CPU, memory, and network usage** before selecting instance size
- Oversized instances increase **monthly AWS bill** without benefit
- Smaller instances give **same performance** for low-traffic applications
- Best for:
 - Development servers
 - Testing environments
 - Small websites
- Always match instance size with **workload requirement**

Example : change the instance type of instance after stopping instance —>action —> change instance type

Change instance type [Info](#) [Get advice](#)

You can change the instance type only if the current instance type and the instance type that you want are compatible.

Instance ID
i-0d0b90810cfa3c2f9 (cost-op-ec2)

Current instance type
c7i-flex.large

New instance type
Q t3.micro X

Change instance type to: t3.micro (32-bit) / (64-bit) architecture can be chosen.

	c7i-flex.large	t3.micro
On-Demand Linux pricing	0.0848 USD per Hour	0.0104 USD per Hour
On-Demand Windows pricing	0.1722 USD per Hour	0.0196 USD per Hour
vCPUs	2 (1 core)	2 (1 core)
Memory (MiB)	4096	1024
Storage (GB)	—	—
Supported root device types	ebs	ebs
Network performance	Up to 12.5 Gigabit	Up to 5 Gigabit
Architecture	x86_64	x86_64

—> and save the changes this will optimise the cost of instance

Instance type changed successfully

Instances (1/3) Info Last updated 3 minutes ago Connect Instance state Actions

Find Instance by attribute or tag (case-sensitive) All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	myec22.lb	i-08a86882c60d3a475	Stopped	t3.micro	—	View alarms +	us-east-1b	—
☐	my-ec21-lb	i-05cafd4b15d784e6	Stopped	t3.micro	—	View alarms +	us-east-1a	—
☒	cost-op-ec2	i-0d0b90810cfa3c2f9	Stopped	t3.small	—	View alarms +	us-east-1f	—

- Result:
 - Same application works
 - **Less cost**
 - Better resource utilization

—> “Right-sizing EC2 means selecting the correct instance size so we don’t pay extra for unused CPU and memory.”

b) Stop Unused Instances

- Stop **dev/test** servers when not in use
- Stopped instance = **no compute cost**

—> Stopping unused EC2 instances helps **reduce unnecessary AWS cost**.

Points:

- Development and testing servers are **not required 24/7**
- Stop EC2 instances when:
 - Office hours are over
 - Work is completed
 - Application is not in use
- When an instance is **stopped**:
 - **No EC2 compute cost**
 - Only storage (EBS) cost is charged
- Very useful for:
 - Dev environments
 - Test environments
 - Temporary projects
- Restart instance anytime when needed
- Helps avoid **wasting money on idle servers**

Instance type changed successfully

Instances (1/3) Info

Last updated 3 minutes ago

Connect

Instance state

Actions

Find Instance by attribute or tag (case-sensitive)

All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
☐	myec22.lb	i-08a86882c60d3a475	Stopped	t3.micro	–	View alarms +	us-east-1b	–
☐	my-ec21-lb	i-05cafd4b15d784e6	Stopped	t3.micro	–	View alarms +	us-east-1a	–
☒	cost-op-ec2	i-0d0b90810cfa3c2f9	Stopped	t3.small	–	View alarms +	us-east-1f	–

Example:

- Dev EC2 runs from **10 AM – 6 PM**
- Stop instance after work hours
- Restart next day
- Result:
 - Same work
 - **Lower monthly AWS bill**

Best Practice:

- Create a habit of stopping instances daily
- Use automation to stop/start instances on schedule

—> “Stopping unused EC2 instances saves cost because AWS does not charge compute cost for stopped instances.”

c) Use Spot Instances

- Very cheap (up to 90% less)
- Best for:
 - Testing
 - Batch jobs
 - Non-critical workloads

Purchasing option | [Info](#)

☐ None

☐ Capacity Blocks

Launch instances for your active capacity blocks

☐ Interruptible Capacity Reservations

Launch instances for your interruptible Capacity Reservations

☒ Spot instances

Request Spot Instances at the Spot price, capped at the On-Demand price

[Discard Spot instance options](#)

—> EBS & Snapshot Cost Optimization

—>Delete **unused EBS volumes**

—>Remove **old snapshots**

—>Use **Lifecycle Manager** to auto-delete old snapshots

—>Choose correct volume type:

- gp3 instead of gp2 (cheaper)

a) Delete **unused EBS volumes**

Unused EBS volumes continue to **generate cost even if no EC2 instance is using them.**

When an EC2 instance is **terminated**, sometimes its EBS volume becomes **detached**.

Detached EBS volume means:

- No instance is using it
- Data is not required anymore

Even if the volume is unused, **AWS still charges for storage**

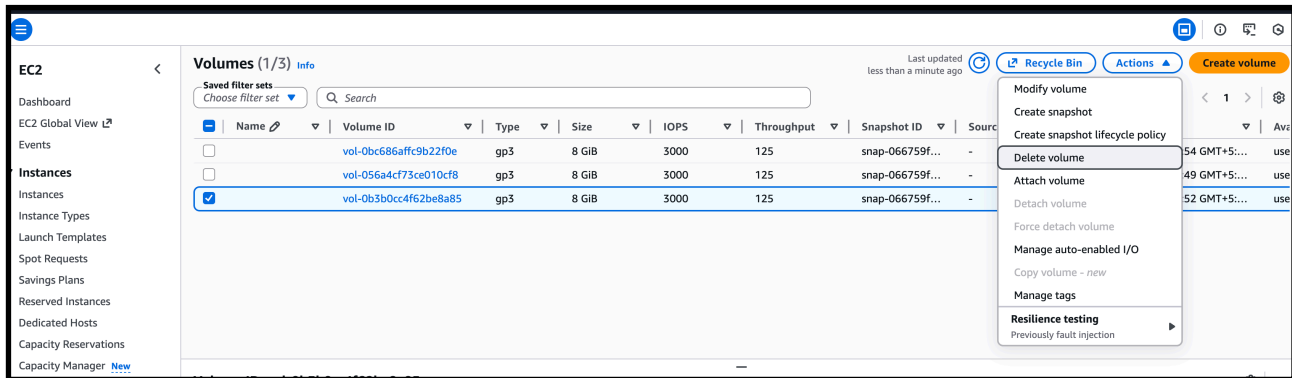
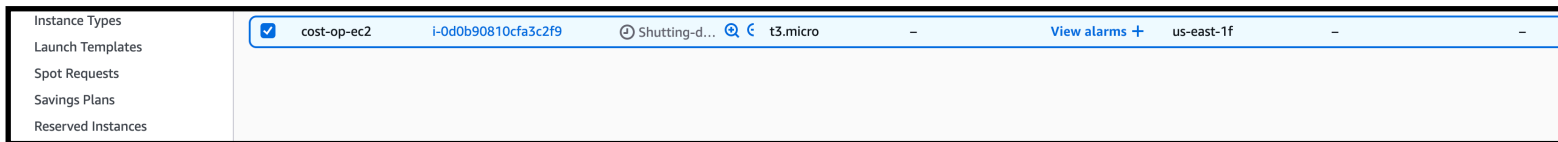
Such volumes should be **identified and deleted**

Deleting unused EBS volumes helps:

- Reduce storage cost
- Avoid unnecessary monthly billing

Example:

- EC2 instance is terminated
- Root or additional EBS volume becomes **detached**
- Volume is no longer required
- Delete the volume



- **Result:** “Deleting unused EBS volumes saves cost because AWS charges for storage even when the volume is not attached to any instance.”
 - Storage removed
 - **EBS cost saved**

Best Practices:

- Regularly check for **available (detached) volumes**
- Delete volumes that are:
 - Not attached to any instance
 - Not needed for backup
- Keep only required volumes

B) Remove old snapshots

EBS snapshots are stored in S3 and **cost money as long as they exist**, even if the instance is deleted.

Points:

- Snapshots are usually taken for:
 - Backup
 - Recovery
- Over time, **old snapshots are no longer required**
- Even if:
 - EC2 instance is terminated
 - EBS volume is deleted
—> Snapshots still remain and generate cost
- Unused and outdated snapshots should be **identified and deleted**
- Removing old snapshots helps:
 - Reduce storage cost
 - Keep backup clean and manageable

Example:

- Daily snapshots are taken
- Snapshots older than **30 or 60 days** are not needed
- Delete old snapshots

The screenshot shows the AWS Management Console interface for the 'Snapshots' section. A green banner at the top indicates 'Successfully created snapshot snap-0e1ae2be4a86833b9'. The 'Snapshots (2/2)' table lists two snapshots:

	Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier
☒		snap-0e1ae2be4a86833b9	1.62 GiB	8 GiB	–	Standard
☒		snap-05c872fb5234c41e5	1.65 GiB	8 GiB	–	Standard

The 'Actions' dropdown menu is open, showing options: 'Create volume from snapshot', 'Create image from snapshot', 'Copy snapshot', 'Launch copy duration calculator', 'Delete snapshot', 'Manage tags', 'Snapshot settings', and 'Archiving'.

- Result:
 - Less storage usage
 - **Lower AWS bill**

Best Practices:

- Keep only **latest required snapshots**
- Delete snapshots of:
 - Old projects
 - Terminated instances
- Use **EBS Lifecycle Manager** to:
 - Automatically delete old snapshots
 - Retain only recent backups

—> “Removing old EBS snapshots saves cost because AWS charges for snapshot storage even when the volume or instance is deleted.”

c) Choosing the correct EBS volume type helps **reduce storage cost without affecting performance.**

Points:

- gp2 and gp3 are **General Purpose SSD** volumes
- gp3 is **cheaper than gp2**
- gp3 gives:
 - Better performance control
 - Lower cost per GB
- In gp2, performance is linked to volume size
- In gp3, performance is **independent of volume size**

- For most workloads:
 - **gp3** provides same or better performance
 - at **lower cost**
- Recommended for:
 - Web servers
 - Application servers
 - Dev and test environm

Example:

- 100 GB volume:
 - gp2 → higher cost
 - gp3 → lower cost with same performance

—-> steps

Go to volume select the volume and—> modify the volume.
—> change the volume type to gp3

EC2 > Volumes > vol-0bc686affc9b22f0e > Modify volume

Modify volume [Info](#)

Modify the type, size, and performance of an EBS volume.

Volume details

Volume ID
vol-0bc686affc9b22f0e

Volume type [Info](#)

General Purpose SSD (gp3) ▲

General Purpose SSD (gp3) ✓

General Purpose SSD (gp2)

Provisioned IOPS SSD (io1)

Provisioned IOPS SSD (io2)

Magnetic (standard)

Min: 3000 IOPS, Max: 80000 IOPS.

Throughput (MiB/s) [Info](#)

125

Min: 125 MiB, Max: 2000 MiB. Baseline: 125 MiB/s.

Result :

- No impact on application
- **Monthly EBS cost reduced**

Best Practice:

- Use gp3 as **default volume type**
- Migrate existing gp2 volumes to gp3

—> “Using gp3 instead of gp2 reduces EBS cost because gp3 is cheaper and gives better performance flexibility.”

—> **S3 Cost Optimization**

AWS S3 STORAGE CLASSES					
Features	S3-Standard	S3-Standard IA	S3 One Zone IA	S3 - RRS	
Durability	99.999999999%	99.999999999%	99.999999999% (Single AZ)	99.99%	99.9%
Availability	99.99%	99.9%	99.5%	99.99%	
Availability Zones	>=3	>=3	1	1	
Minimum Storage Duration	Unlimited	30 Days	30 Days	Unlimited	
Access Speed (Latency)	Miliseconds	Miliseconds	Miliseconds	Miliseconds	1 m
Data Retrieval Fee	Nil	Per GB Data Retrived	Per GB Data Retrived	Nil	Pe
Minimum Billable Object Size	NA	128 KB (the smaller objects will be charged for 128KB)	128 KB (the smaller objects will be charged for 128KB)	NA	
Type of Storage	Object	Object	Object	Object	
Lifecycle Management Policies	Available	Available	Available	Available	A
SSL Support	Yes	Yes	Yes	Yes	
Amazon S3 SLA Backing	99.9%	99%	99%	Data Not Available	

—> a) Cost Optimization Action Performed on S3

Action: Changed storage class of object to cheaper option

Points:

- Initially, the object was stored in **S3 Standard**
- S3 Standard is used for **frequently accessed data**

The screenshot shows the Amazon S3 console interface for a bucket named 'my-s3-bucket-889'. The breadcrumb navigation at the top reads: Amazon S3 > Buckets > my-s3-bucket-889 > peakpx-2.jpg. The interface is divided into two main sections: 'Bucket properties' on the left and 'Management configurations' on the right.

Bucket properties

- Bucket Versioning**: When enabled, multiple variants of an object can be stored in the bucket to easily recover from unintended user actions and application failures. Status: **Disabled**. A warning box states: "Bucket 'my-s3-bucket-889' doesn't have Bucket Versioning enabled. We recommend that you enable Bucket Versioning to help protect against unintentionally overwriting or deleting objects. [Learn more](#)". An 'Enable Bucket Versioning' button is present.
- Storage class**: Amazon S3 offers a range of storage classes designed for different use cases. [Learn more](#) or see [Amazon S3 pricing](#). The current storage class is 'Standard'. An 'Edit' button is available.
- Server-side encryption settings**: Server-side encryption protects data at rest. The encryption type is 'Amazon S3 managed keys (SSE-S3)'. An 'Edit' button is available.

Management configurations

- Replication status**: When a replication rule is applied to an object the replication status indicates the progress of the operation. Status: '-'. A 'View replication rules' link is present.
- Expiration rule**: You can use a lifecycle configuration to define expiration rules to schedule the removal of this object after a pre-defined time period. Status: '-'. An 'Expiration date' section states: 'The object will be permanently deleted on this date.' Status: '-'.

- The file was **not accessed regularly**
- Keeping it in S3 Standard was **unnecessarily increasing cost**
- To reduce cost, the storage class was changed to **Glacier Deep Archive**

The screenshot shows the 'Edit storage class' dialog in the Amazon S3 console for bucket 'my-s3-bucket-889'. The breadcrumb navigation is: Amazon S3 > Buckets > my-s3-bucket-889 > Edit storage class. A note states: 'This action creates a copy of the object with updated settings and a new last-modified date. You can change the storage class without making a new copy of the object using a [lifecycle rule](#). [View copy restrictions and limitations](#)'.

Storage class

Amazon S3 offers a range of storage classes designed for different use cases. [Learn more](#) or see [Amazon S3 pricing](#).

	Storage class	Designed for	Availability Zones	Min storage duration	Min billable object size	Monitoring and auto-tiering fees	Retrieval fees
☐	Standard	Frequently accessed data (more than once a month) with milliseconds access	≥ 3	-	-	-	-
☐	Intelligent-Tiering	Data with changing or unknown access patterns	≥ 3	-	-	Per-object fees apply for objects >= 128 KB	-
☐	Standard-IA	Infrequently accessed data (once a month) with milliseconds access	≥ 3	30 days	128 KB	-	Per-GB fees apply
☐	One Zone-IA	Recreateable, infrequently accessed data (once a month) with milliseconds access	1	30 days	128 KB	-	Per-GB fees apply
☐	Glacier Instant Retrieval	Long-lived archive data accessed once a quarter with instant retrieval in milliseconds	≥ 3	90 days	128 KB	-	Per-GB fees apply
☐	Glacier Flexible Retrieval (formerly Glacier)	Long-lived archive data accessed once a year with retrieval of minutes to hours	≥ 3	90 days	-	-	Per-GB fees apply
☒	Glacier Deep Archive	Long-lived archive data accessed less than once a year with retrieval of hours	≥ 3	180 days	-	-	Per-GB fees apply
☐	Reduced redundancy	Noncritical, frequently accessed data with milliseconds access (not recommended as S3 Standard is more cost effective)	≥ 3	-	-	-	Per-GB fees apply

Specified objects

Find objects by name

- Glacier Deep Archive is used for:
 - Long-term storage
 - Rarely accessed data
- After changing the storage class:
 - Storage cost was **significantly reduced**
 - Data is still safely stored
 - Only retrieval time is higher, which is acceptable for archive data

Result:

- No data loss
- Lower S3 monthly bill
- Effective cost optimization achieved

—>“I optimized S3 cost by moving rarely accessed data from S3 Standard to Glacier Deep Archive, which reduced storage cost.”

—> b) Enable Lifecycle Policies (S3 Cost Optimization)

Cost Optimization Action Performed: Enabled Lifecycle Policy

Points:

- Data stored in S3 **keeps increasing over time**
- Old files are **not accessed frequently**
- Keeping old data in **S3 Standard increases cost**
- To optimize cost, an **S3 Lifecycle Policy** was enabled

- Lifecycle policy automatically manages storage class based on age
- No manual work is required after configuration

Example:

- Objects stored in S3 Standard
- Lifecycle rule configured:
 - After **30 days** → **move data to Glacier**
- Result:
 - Storage automatically moved to cheaper class
 - No data loss
 - Reduced S3 storage cost

Lifecycle rule actions

Choose the actions you want this rule to perform.

☒ **Transition current versions of objects between storage classes**
This action will move current versions.

☐ **Transition noncurrent versions of objects between storage classes**
This action will move noncurrent versions.

☐ **Expire current versions of objects**

☐ **Permanently delete noncurrent versions of objects**

☐ **Delete expired object delete markers or incomplete multipart uploads**
These actions are not supported when filtering by object tags or object size.

⚠ Transitions are charged per request
For a lifecycle transition action, each request corresponds to an object transition. For details on lifecycle transition pricing, see requests pricing info on the [Storage & requests](#) tab of the [Amazon S3 pricing page](#).

☒ I acknowledge that this lifecycle rule will incur a transition cost per request.

ⓘ By default, objects less than 128KB will not transition across any storage class
We don't recommend transitioning objects less than 128 KB because the transition costs can outweigh the storage savings. If your use case requires transitioning objects less than 128 KB, specify a minimum object size filter for each applicable lifecycle rule with a transition action.

Transition current versions of objects between storage classes

Choose transitions to move current versions of objects between storage classes based on your use case scenario and performance access requirements. These transitions start from when the objects are created and are consecutively applied. [Learn more](#)

Choose storage class transitions

Days after object creation

Glacier Deep Archive

30

Remove

Add transition

S3 Lifecycle Policy – Cost Optimization (Current Versions)

Points:

- Newly uploaded objects are stored in **S3 Standard** for immediate access
- After some time, these objects are **rarely accessed**
- Keeping such data in S3 Standard increases storage cost

- A lifecycle policy was configured to manage this automatically
- The rule transitions **current versions of objects** to **Glacier Deep Archive**
- Transition happens **30 days after object creation**
- Glacier Deep Archive is the **lowest-cost S3 storage class**
- This significantly reduces storage cost for long-term data
- No manual intervention is required after configuration

Result:

- Automatic movement of old data to cheaper storage
- Reduced S3 monthly bill
- Efficient long-term data storage

—> configured an S3 lifecycle rule to move current object versions to Glacier Deep Archive after 30 days to reduce storage cost.

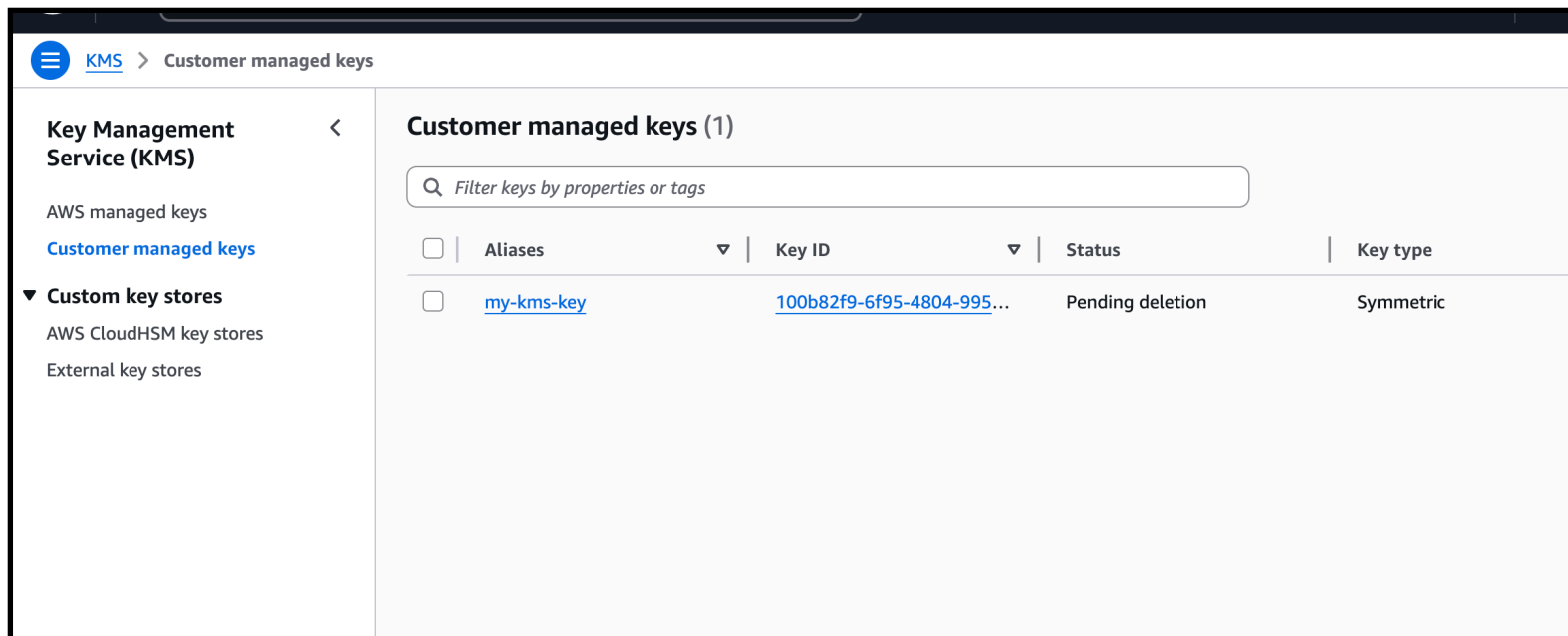
—> AWS KMS Cost Optimization – Deleting Unused KMS Keys

Cost Optimization Action: Deleted Unused KMS Key

Points:

- AWS KMS charges **monthly cost for each customer-managed key**
- Some KMS keys are created for:
 - Testing
 - Temporary projects
- After the project is completed, the KMS key is **no longer required**
- Keeping unused KMS keys **continues to generate cost**

- To optimize cost, the **unused KMS key was scheduled for deletion**
- After deletion:
 - AWS **stops charging** for that KMS key
 - Unnecessary security resources are removed



What Happens When a KMS Key Is Deleted

Important Points:

- KMS keys are **not deleted immediately**
- They are **scheduled for deletion** (minimum 7 days, maximum 30 days)
- During the waiting period:
 - Key cannot be used for encryption
 - Key can be restored if needed
- After deletion is completed:
 - Key is **permanently removed**
 - Data encrypted with that key **cannot be decrypted**
- AWS **does not charge** for deleted KMS keys

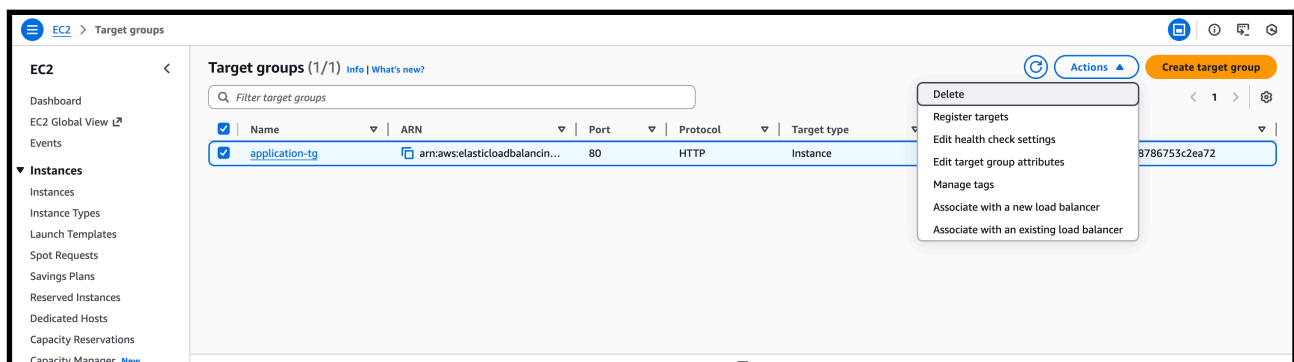
Result (Cost Optimization Outcome)

- Reduced KMS monthly cost
- Removed unused security resources
- Cleaner and cost-efficient AWS environment

—> Cost was optimized by deleting unused KMS keys, which stopped unnecessary monthly KMS charges.

—> Load Balancer Cost Optimization

- Use **Application Load Balancer** only when required
- Delete unused target groups



- Target groups are created for:
 - Load Balancers
 - Testing
 - Old applications
- Over time, some target groups become **unused**
- These target groups:

- Are **not attached to any Load Balancer**
- Do not receive traffic
- Keeping unused target groups **creates management overhead**
- To optimize resources, unused target groups were **identified and deleted**
- Deleting unused target groups helps:
 - Clean up AWS environment
 - Avoid confusion during configuration
 - Improve resource management

What Happens After Deleting Target Groups

Points:

- No application impact if target group is **not attached**
- Load Balancer continues to work normally
- Unused backend configurations are removed
- AWS environment becomes **clean and optimized**

—> Unused target groups that were not attached to any load balancer were deleted to clean up resources and improve cost and resource management.

- Remove idle load balancers

—> **Best Practices Summary**

- > Delete unused resources
- > Right-size EC2 & RDS
- > Use Auto Scaling
- > Use S3 lifecycle rules
- > Monitor bills daily
- > Set AWS Budgets

Conclusion :

AWS Cost Optimization helps organizations **reduce cloud cost without affecting performance**. By choosing the right service size, removing unused resources, and monitoring usage, we can **save a lot of money in AWS**.